

Un solide MPM avec endommagement et rupture, couplé au fluide APIC

Tutoriel en deux parties : le pont seul, puis le pont sous l'eau

Résumé

Ce tutoriel ajoute au solveur APIC 2D du tutoriel précédent une **phase solide** capable de se déformer, de s'endommager et de se rompre : un pont, un tablier, un mur. La méthode est **MLS-MPM** (*Material Point Method*), qui partage avec APIC la même grille, les mêmes poids d'interpolation et le même cycle $P2G \rightarrow \text{grille} \rightarrow G2P$. Ce qui change tient en trois ingrédients portés par chaque particule solide : un **gradient de déformation $\mathbf{F}$** et une loi élastique, une **variable d'endommagement $D \in [0, 1]$** qui réduit la raideur, et un état **rompu** qui supprime la cohésion. La partie I construit et teste le solide *seul* (une poutre encastree que l'on charge jusqu'à la ruine), la partie II le couple au fluide sur une grille commune (un bloc d'eau qui tombe sur le tablier). Le code complet, testé avec Taichi 1.7.4, est donné en annexe et vit dans le dossier MpM/ du projet.

Table des matières

1	Prérequis et organisation des fichiers	2
2	L'idée générale	2
2.1	MPM, c'est APIC avec une mémoire de la déformation	2
2.2	Trois niveaux de modèle	3
2.3	Par rapport au code MPM classique	3
3	Le minimum de mathématiques	3
3.1	Le gradient de déformation	3
3.2	Décomposition polaire et allongements principaux	4
3.3	La loi de comportement	4
3.4	La force nodale MLS-MPM	4
3.5	L'endommagement	5
3.6	La rupture : supprimer la cohésion	5
3.7	Stabilité et choix des paramètres	6
I	Le solide seul	6
4	Étape 1 : champs et loi de comportement	6
5	Étape 2 : P2G solide	7
6	Étape 3 : grille et encastrement	7
7	Étape 4 : G2P solide, version élastique	8
8	Étape 5 : endommagement	9
9	Étape 6 : rupture	9

10	Étape 7 : le programme de démonstration	10
II	Couplage avec le fluide APIC	11
11	Le principe : une grille, deux phases	11
11.1	Pourquoi c'est presque gratuit	11
11.2	L'ordre des kernels	11
11.3	Un seul pas de temps	11
12	Le programme couplé : <code>main_fsi.py</code>	12
12.1	Scène	12
12.2	Champs	13
12.3	Initialisation et advection du fluide	13
12.4	Le pas de temps couplé	14
12.5	Affichage et boucle	14
12.6	Ce que l'on doit voir	14
13	Limites et pour aller plus loin	15
A	Code complet : <code>MpM/mpm_solid.py</code>	15
B	Code complet : <code>MpM/demo_solid.py</code>	18
C	Code complet : <code>main_fsi.py</code>	21

1 Prérequis et organisation des fichiers

On suppose acquis le tutoriel APIC (poids B-spline quadratiques, `base/fx/w`, transfert affine C_p , formule $D_p^{-1} = 4/\Delta x^2$). Le projet est organisé ainsi :

Fichier	Rôle
APIC/APIC.py	fluide : P2G, <code>grid_step</code> , G2P (inchangé)
MpM/mpm_solid.py	solide : P2G_solid, <code>grid_update</code> , G2P_solid, utilitaires
MpM/demo_solid.py	partie I : la poutre seule, avec curseur de charge
main_fsi.py	partie II : eau + pont sur la grille commune

Lancer depuis la racine du projet :

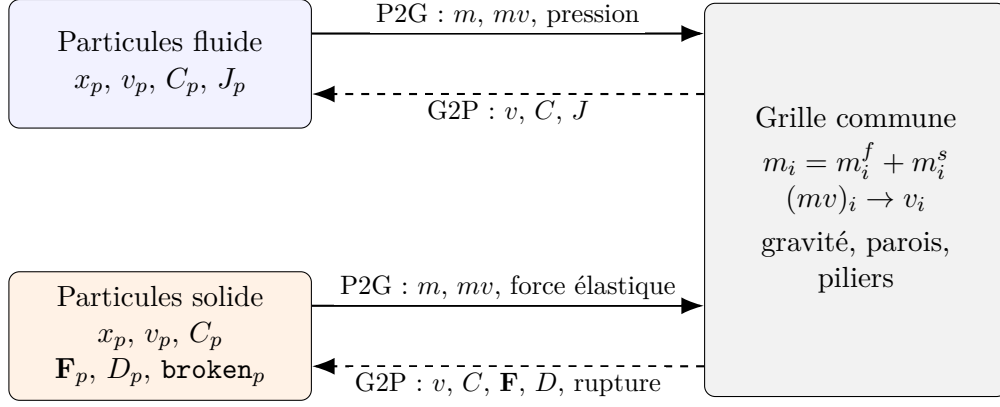
```
python -m MpM.demo_solid      # partie I : la poutre seule
python main_fsi.py           # partie II : eau + pont
```

Touches communes : **Espace** bascule la couleur des particules solides (endommagement / déformation), **R** réinitialise, **Échap** quitte. Dans la partie I, les touches **1**, **2**, **3** choisissent le modèle (élastique, +endommagement, +rupture) et un curseur règle la charge.

2 L'idée générale

2.1 MPM, c'est APIC avec une mémoire de la déformation

Une particule de fluide APIC transporte (x_p, v_p, C_p, J_p) : sa seule mémoire de l'histoire est le scalaire J_p , taux de dilatation, qui suffit à définir une pression. Un solide, lui, résiste à *toute* déformation, pas seulement au changement de volume : cisaillement, flexion, étirement. Il faut donc une mémoire tensorielle, le **gradient de déformation** $\mathbf{F}_p$ (2×2), et une loi qui transforme $\mathbf{F}_p$ en contrainte $\boldsymbol{\sigma}_p$ (2×2). Le reste du cycle est identique :



2.2 Trois niveaux de modèle

1. **Élastique** : $\mathbf{F}_p$ est mis à jour à chaque pas, une loi hyperélastique donne la contrainte, la force interne est déposée sur la grille. La poutre fléchit, vibre, revient.
2. **Endommagement** : une variable D_p croît de façon irréversible quand l'allongement dépasse un seuil ; la raideur devient $(1 - D_p)E$. La poutre s'assouplit là où elle est trop sollicitée.
3. **Rupture** : quand $D_p = 1$ la particule perd toute cohésion : elle ne transmet plus de traction, seulement de la compression. Les morceaux se séparent et tombent.

On implémente les trois d'un coup dans `G2P_solid` avec deux drapeaux `use_damage` et `use_rupture`, pour pouvoir les comparer au clavier.

2.3 Par rapport au code MPM classique

Le code MPM « éléments Q4 » que vous avez peut-être sous les yeux (fonctions de forme de Lagrange, `nodal_iforces == Vp * stress @ dNdx`, contrainte incrémentée par `dsigma = C @ dEps`) fait la même chose que nous, avec trois différences de forme :

	MPM classique (Q4)	MLS-MPM (ce tutoriel)
Fonctions de forme	Lagrange bilinéaires, 2×2 nœuds	B-spline quadratique, 3×3 nœuds
Gradient des poids	$\partial N_I / \partial x$ calculé via le jacobien	$\nabla w_{ip} \approx \frac{4}{\Delta x^2} w_{ip} (x_i - x_p)$
Gradient de vitesse	$L_p = \sum_I v_I \nabla N_I^T$	C_p (déjà calculé pour APIC)
Contrainte	hypoélastique : $\Delta \boldsymbol{\sigma} = \mathbf{C} \Delta \boldsymbol{\epsilon}$	hyperélastique : $\boldsymbol{\sigma} = \boldsymbol{\sigma}(\mathbf{F})$

La formulation hyperélastique (contrainte fonction de $\mathbf{F}$ seul) est plus robuste aux grandes rotations, ce qui compte pour des morceaux de pont qui tombent en tournant. Et $C_p \approx \nabla v$ est *gratuit* : APIC le calcule déjà.

3 Le minimum de mathématiques

3.1 Le gradient de déformation

Une particule solide occupait la position $\mathbf{X}_p$ au repos et occupe $\mathbf{x}_p$ maintenant. Le gradient de déformation $\mathbf{F} = \partial \mathbf{x} / \partial \mathbf{X}$ décrit comment un petit voisinage a été étiré et tourné : $\mathbf{F} = \mathbf{I}$ au repos, $\det \mathbf{F} = J$ est le rapport de volume (le J_p du fluide!). Son évolution suit

$$\dot{\mathbf{F}} = (\nabla v) \mathbf{F},$$

et comme $C_p \approx \nabla v$ au voisinage de la particule, la mise à jour discrète est simplement

$$\mathbf{F}_p \leftarrow (\mathbf{I} + \Delta t C_p) \mathbf{F}_p$$

C'est la généralisation matricielle de $J_p \leftarrow J_p(1 + \Delta t \operatorname{tr} C_p)$ du fluide (prendre le déterminant des deux côtés redonne cette formule au premier ordre).

3.2 Décomposition polaire et allongements principaux

Toute matrice $\mathbf{F}$ se décompose en une rotation et un étirement pur : $\mathbf{F} = \mathbf{R} \mathbf{S}$ avec $\mathbf{R}$ orthogonale et $\mathbf{S}$ symétrique. Numériquement on passe par la SVD $\mathbf{F} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$, d'où

$$\mathbf{R} = \mathbf{U} \mathbf{V}^\top, \quad \mathbf{\Sigma} = \text{diag}(s_1, s_2).$$

Les s_k sont les **allongements principaux** : $s_k = 1$ au repos, $s_k > 1$ en traction, $s_k < 1$ en compression, dans deux directions orthogonales. Taichi fournit `ti.svd(F)` pour les matrices 2×2 et 3×3 ; on l'utilise partout.

3.3 La loi de comportement

On utilise le modèle **corotationnel** (*fixed corotated*, Stomakhin et al. 2012), le standard des solveurs MPM graphiques car il reste stable en grandes rotations et même en cas d'inversion locale de $\mathbf{F}$:

$$\mathbf{P} = 2\mu (\mathbf{F} - \mathbf{R}) + \lambda J (J - 1) \mathbf{F}^{-\top}.$$

$\mathbf{P}$ est la contrainte de Piola-Kirchhoff (par unité de surface de repos) ; celle qui intervient dans la force nodale est $\boldsymbol{\tau} = \mathbf{P} \mathbf{F}^\top$ (Kirchhoff), qui se simplifie en

$$\boxed{\boldsymbol{\tau} = 2\mu (\mathbf{F} - \mathbf{R}) \mathbf{F}^\top + \lambda J (J - 1) \mathbf{I}}$$

Le premier terme est la pénalité de *distorsion* (écart entre $\mathbf{F}$ et sa rotation), le second celle de *volume* : c'est exactement la loi de pression du fluide, $E(J - 1)\mathbf{I}$, avec λJ à la place de E . Les coefficients de Lamé viennent du module d'Young E et du coefficient de Poisson ν :

$$\mu = \frac{E}{2(1 + \nu)}, \quad \lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)}.$$

Une alternative classique est le Néo-Hookéen, $\boldsymbol{\tau} = \mu(\mathbf{F}\mathbf{F}^\top - \mathbf{I}) + \lambda \ln J \mathbf{I}$, qui a les mêmes petites déformations mais explose si $J \rightarrow 0$; on le mentionne pour l'exercice.

3.4 La force nodale MLS-MPM

Avec la contrainte de Cauchy $\boldsymbol{\sigma} = \boldsymbol{\tau}/J$ et le volume courant $V_p = J V_p^0$, la force exercée par la particule sur le nœud i est

$$\mathbf{f}_i = - \sum_p V_p \boldsymbol{\sigma}_p \nabla w_{ip} = - \sum_p V_p^0 \boldsymbol{\tau}_p \nabla w_{ip},$$

et avec l'approximation MLS du gradient des poids, $\nabla w_{ip} \approx \frac{4}{\Delta x^2} w_{ip} (x_i - x_p)$,

$$\Delta t \mathbf{f}_i = w_{ip} \underbrace{\left(- \Delta t \frac{4V_p^0}{\Delta x^2} \boldsymbol{\tau}_p \right)}_{\text{matrice } 2 \times 2} (x_i - x_p).$$

Comparez au fluide : `stress * dpos` avec `stress scalaire`, car $\boldsymbol{\tau} = E(J - 1)\mathbf{I}$ y est isotrope. Ici c'est une matrice appliquée à `dpos`. Comme le terme APIC $m_p C_p (x_i - x_p)$ a exactement la même forme, on additionne les deux matrices en une seule, appelée **affine** dans le code :

$$(mv)_i += w_{ip} \left(m_p v_p + \underbrace{\left(- \Delta t \frac{4V_p^0}{\Delta x^2} \boldsymbol{\tau}_p + m_p C_p \right)}_{\text{affine}} (x_i - x_p) \right).$$

3.5 L'endommagement

Critère. Un pont casse en *traction* (fibre inférieure à mi-portée, fibre supérieure aux appuis), pas en compression. On mesure donc l'**allongement principal maximal**

$$\varepsilon_p = \max(s_1, s_2) - 1,$$

directement lu sur la SVD de $\mathbf{F}_p$. C'est un critère de type Rankine (contrainte principale maximale) écrit en déformation. Le critère de Von Mises, $\sigma_{eq} = \sqrt{\frac{3}{2} \mathbf{s} : \mathbf{s}}$ avec $\mathbf{s}$ le déviateur de $\boldsymbol{\sigma}$, est le bon choix pour la *plasticité* des métaux, mais il est symétrique traction/compression : un pilier comprimé s'endommagerait autant qu'un tablier tendu. Pour un matériau fragile (béton, pierre, verre) le critère en traction est plus fidèle.

Loi d'évolution. Deux seuils, ε_0 (début de l'endommagement) et ε_f (rupture), et une interpolation linéaire entre les deux :

$$D_p \leftarrow \max\left(D_p, \text{clamp}\left(\frac{\varepsilon_p - \varepsilon_0}{\varepsilon_f - \varepsilon_0}, 0, 1\right)\right).$$

Le max rend D_p **irréversible** : une fissure ne se referme pas. La raideur effective est

$$E_{\text{eff}} = (1 - D_p) E \quad \Longleftrightarrow \quad \mu_{\text{eff}} = (1 - D_p) \mu, \quad \lambda_{\text{eff}} = (1 - D_p) \lambda.$$

C'est un *adoucissement* : plus la particule est endommagée, moins elle résiste, donc plus elle s'allonge, donc plus D_p croît. La déformation se localise d'elle-même dans une bande étroite : c'est la fissure.

3.6 La rupture : supprimer la cohésion

Marquer simplement **broken** = 1 et mettre la contrainte à zéro ne suffit pas : la particule continuerait de partager des nœuds avec ses voisines et resterait « collée » au reste par le champ de vitesse commun. Il faut lui donner un comportement de matériau **sans cohésion** (du sable) : il résiste à la compression, jamais à la traction. Concrètement, à chaque pas et pour chaque particule rompue :

1. à l'instant de la rupture, $\mathbf{F}_p \leftarrow \mathbf{I}$: la particule oublie la déformation accumulée (l'énergie élastique de traction est perdue, comme dans une vraie fissure) ;
2. après la mise à jour $\mathbf{F}_p \leftarrow (\mathbf{I} + \Delta t C_p) \mathbf{F}_p$, on écrête les allongements principaux :

$$s_k \leftarrow \text{clamp}(s_k, 0.1, 1), \quad \mathbf{F}_p \leftarrow \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T.$$

Un étirement $s_k > 1$ est immédiatement ramené à 1 : aucune traction ne peut naître. La borne inférieure évite l'inversion sous un écrasement extrême ;

3. la contrainte est calculée avec la raideur *pleine* ($k = 1$, pas $1 - D$), sinon la particule ne résisterait même plus à la compression et les morceaux s'interpénétreraient.

C'est le même mécanisme que la plasticité de la neige de Stomakhin et al. (2013), poussé à l'extrême (aucune traction admise). Il ne coûte qu'une SVD par particule rompue.

Limite à connaître. Deux morceaux séparés par moins d'une cellule interagissent encore à travers les nœuds partagés (ils « se voient » par la grille). Les fissures ont donc une largeur minimale d'environ une cellule, et deux fragments en contact glissent sans frottement bien défini. Les remèdes classiques sont CPIC (*compatible PIC* : deux jeux de vitesses nodales de part et d'autre d'une fissure) ou un réseau explicite de liaisons entre particules initialement voisines (cassées quand leur allongement $\epsilon_{pq} = |\mathbf{x}_p - \mathbf{x}_q|/d_{pq}^0 - 1$ dépasse un seuil). Pour un premier solveur, le modèle continu ci-dessus donne déjà des fissures nettes.

3.7 Stabilité et choix des paramètres

Le schéma est explicite : $\Delta t < \Delta x / c_p$ avec la vitesse des ondes de compression

$$c_p = \sqrt{\frac{\lambda + 2\mu}{\rho_s}}.$$

Dans le code, $\Delta t = 0.4 \Delta x / c_p$ est *calculé* à partir des paramètres, pas choisi à la main. Valeurs de la partie I :

Paramètre	Valeur	Rôle
n	128	nœuds par côté, $\Delta x = 1/128$
ρ_s	2	masse volumique (2 fois l'eau : les morceaux coulent)
E, ν	3000, 0.3	$\mu = 1154, \lambda = 1731, c_p \approx 45$
Δt	$0.4 \Delta x / c_p \approx 7 \cdot 10^{-5}$	calculé
$\varepsilon_0, \varepsilon_f$	0.05, 0.20	seuils d'endommagement et de rupture
V_p	$(\Delta x / 2)^2$	4 particules par cellule, sur un réseau régulier
poutre	$[0.15, 0.85] \times [0.55, 0.61]$	extrémités noyées dans des piliers de largeur 0.06

$E = 3000$ est *très* mou pour un vrai matériau (~ 30 GPa pour du béton), mais c'est ce qui rend la flèche visible à l'œil ; de même $\varepsilon_f = 0.2$ est ductile. L'ordre de grandeur qui compte est la déformation de flexion d'une poutre encastree sous son poids, $\varepsilon \approx \rho_s g L^2 / (2hE)$: avec $L = 0.58$, $h = 0.06$, on trouve $\varepsilon \approx 0.02$ par unité de charge, à comparer aux seuils. Le curseur de charge multiplie g et permet de balayer ε de 0 à 0.1.

Première partie

Le solide seul

4 Étape 1 : champs et loi de comportement

Une particule solide porte, en plus de $x, v, C : \mathbf{F}$ (matrice 2×2), D (réel) et **broken** (entier). La grille est la même que pour le fluide, plus un masque d'obstacles qui servira à encastree la poutre.

```
x_s = ti.Vector.field(2, ti.f32, n_solid)      # position
v_s = ti.Vector.field(2, ti.f32, n_solid)      # vitesse
C_s = ti.Matrix.field(2, 2, ti.f32, n_solid)   # matrice affine APIC (~ grad v)
F_s = ti.Matrix.field(2, 2, ti.f32, n_solid)   # gradient de déformation (I au repos)
D_s = ti.field(ti.f32, n_solid)                # endommagement dans [0, 1]
broken_s = ti.field(ti.i32, n_solid)           # 1 si rompue
col_s = ti.Vector.field(3, ti.f32, n_solid)    # couleur d'affichage

grid_v = ti.Vector.field(2, ti.f32, (n_grid, n_grid))
grid_m = ti.field(ti.f32, (n_grid, n_grid))
mask = ti.field(ti.i32, (n_grid, n_grid))      # 1 = pilier rigide
```

La loi de comportement est une fonction Taichi (@ti.func) callable depuis un kernel. Elle reçoit $\mathbf{F}$ et les coefficients de Lamé (éventuellement déjà réduits par $1 - D$) et renvoie $\boldsymbol{\tau}$:

```
@ti.func
def kirchhoff_stress(F, mu: float, la: float):
    U, sig, V = ti.svd(F)
    R = U @ V.transpose()                    # rotation de la décomposition polaire
    J = F.determinant()
    I = ti.Matrix.identity(ti.f32, 2)
    return 2.0 * mu * (F - R) @ F.transpose() + la * J * (J - 1.0) * I
```

C'est la formule encadrée du §3.3, ligne pour ligne. Pour essayer le Néo-Hookéen, remplacez le `return` par `mu * (F @ F.transpose() - I) + la * ti.log(J) * I`.

5 Étape 2 : P2G solide

Même squelette que le P2G du fluide : stencil, poids, puis dépôt sur les 9 nœuds. Deux différences : la contrainte est une matrice, et le kernel *ne remet pas la grille à zéro* (pour pouvoir s'ajouter au dépôt du fluide en partie II ; en partie I, un petit kernel `clear_grid` s'en charge avant).

```
@ti.kernel
def P2G_solid(grid_m: ti.template(), grid_v: ti.template(),
              x: ti.template(), v: ti.template(), C: ti.template(), F: ti.template(),
              D: ti.template(), broken: ti.template(),
              inv_dx: float, dt: float, dx: float,
              mu: float, la: float, p_mass: float, p_vol: float):
    for p in x:
        base = (x[p] * inv_dx - 0.5).cast(int)
        fx = x[p] * inv_dx - base
        w = [0.5 * (1.5 - fx)**2, 0.75 - (fx - 1.0)**2, 0.5 * (fx - 0.5)**2]

        k = 1.0 - D[p] # raideur effective (1 - D) E
        if broken[p] == 1:
            k = 1.0 # rompue : raideur pleine, F écrêté dans G2P
        tau = kirchhoff_stress(F[p], k * mu, k * la)

        affine = (-dt * p_vol * 4.0 * inv_dx * inv_dx) * tau + p_mass * C[p]

        for i, j in ti.static(ti.ndrange(3, 3)):
            offset = ti.Vector([i, j])
            d_pos = (offset - fx) * dx
            weight = w[i].x * w[j].y
            grid_v[base + offset] += weight * (p_mass * v[p] + affine @ d_pos)
            grid_m[base + offset] += weight * p_mass
```

Ligne par ligne.

- `k` : le facteur $(1 - D_p)$ du §3.5, forcé à 1 pour une particule rompue (§3.6, point 3).
- `affine` : la matrice $-\Delta t \frac{4V_p^0}{\Delta x^2} \tau_p + m_p C_p$. Notez que `p_vol` est le volume de repos V_p^0 : c'est τ (et non σ) qui va avec.
- `affine @ d_pos` remplace le `C[p] @ d_pos` et le `stress * d_pos` du fluide d'un seul coup.

6 Étape 3 : grille et encastrement

La mise à jour de la grille est *identique* à celle du fluide : on la recopie dans `mpm_solid.py` sous le nom `grid_update` pour que la partie I soit autonome (en partie II on utilisera directement `grid_step` du fluide).

```
@ti.kernel
def grid_update(grid_m: ti.template(), grid_v: ti.template(), mask: ti.template(),
               dt: float, g: float, bound: int, n_grid: int):
    for i, j in grid_m:
        if grid_m[i, j] > 0:
            grid_v[i, j] /= grid_m[i, j]
            grid_v[i, j].y -= dt * g
            # parois glissantes du domaine (comme le fluide) ...
            if mask[i, j] == 1:
                grid_v[i, j] = [0.0, 0.0]
```

Encastrement. Un nœud du masque a une vitesse nulle. Une particule dont les 9 nœuds du stencil sont dans le masque ne bouge donc jamais : c'est un encastrement parfait, sans aucun code supplémentaire. Il suffit de *noyer* les extrémités de la poutre dans les piliers :

```
@ti.kernel
def init_mask():
    for i, j in mask:
        pos = ti.Vector([i, j]) * dx
        left = beam_x0 <= pos.x <= beam_x0 + pillar_w
        right = beam_x1 - pillar_w <= pos.x <= beam_x1
        mask[i, j] = 1 if ((left or right) and pos.y <= beam_y1) else 0
```

Les piliers montent jusqu'au *dessus* de la poutre ($\text{pos.y} \leq \text{beam.y1}$) : les particules des extrémités sont entièrement gelées. Pour un appui simple (rotation libre), on arrêterait les piliers sous la poutre.

7 Étape 4 : G2P solide, version élastique

La première moitié est le G2P du fluide (vitesse et C_p). Ensuite, à la place de $J_p \leftarrow J_p(1 + \Delta t \text{ tr } C_p)$, on met à jour $\mathbf{F}_p$. Voici le kernel complet ; les deux blocs `if broken / elif use_damage` sont expliqués aux étapes suivantes et sont inactifs en mode élastique.

```
@ti.kernel
def G2P_solid(grid_v: ti.template(),
              x: ti.template(), v: ti.template(), C: ti.template(), F: ti.template(),
              D: ti.template(), broken: ti.template(),
              inv_dx: float, dt: float, dx: float, bound: int,
              eps0: float, epsf: float, use_damage: int, use_rupture: int):
    I = ti.Matrix.identity(ti.f32, 2)
    for p in x:
        base = (x[p] * inv_dx - 0.5).cast(int)
        fx = x[p] * inv_dx - base
        w = [0.5 * (1.5 - fx)**2, 0.75 - (fx - 1.0)**2, 0.5 * (fx - 0.5)**2]
        v_new = ti.Vector.zero(ti.f32, 2)
        C_new = ti.Matrix.zero(ti.f32, 2, 2)
        for i, j in ti.static(ti.ndrange(3, 3)):
            offset = ti.Vector([i, j])
            d_pos = (offset - fx) * dx
            weight = w[i].x * w[j].y
            g_v = grid_v[base + offset]
            v_new += weight * g_v
            C_new += weight * g_v.outer_product(d_pos) * (4.0 * inv_dx * inv_dx)
        v[p] = v_new
        C[p] = C_new

        # --- gradient de déformation : dF/dt = (grad v) F, grad v ~ C
        F_new = (I + dt * C_new) @ F[p]

        if broken[p] == 1:
            # étape 6 : rupture
            U, sig, V = ti.svd(F_new)
            for d in ti.static(range(2)):
                sig[d, d] = ti.math.clamp(sig[d, d], 0.1, 1.0)
            F_new = U @ sig @ V.transpose()

        elif use_damage == 1:
            # étape 5 : endommagement
            U, sig, V = ti.svd(F_new)
            eps = ti.max(sig[0, 0], sig[1, 1]) - 1.0
            D_new = ti.math.clamp((eps - eps0) / (epsf - eps0), 0.0, 1.0)
            D[p] = ti.max(D[p], D_new)
```

```

if use_rupture == 1 and D[p] >= 1.0:
    broken[p] = 1
    F_new = I

F[p] = F_new
x[p] += dt * v[p]
x[p] = ti.math.clamp(x[p], bound * dx, 1.0 - bound * dx)

```

Ce que l'on doit voir (touche 1, charge 1). La poutre fléchit d'environ une cellule à mi-portée, oscille quelques fois et se stabilise (la dissipation numérique de PIC/APIC amortit les vibrations). Avec le curseur de charge à 3, la flèche triple, sans jamais casser : un solide élastique pur revient toujours. En mode couleur « déformation » (Espace), la fibre inférieure à mi-portée et les fibres supérieures aux encastrement apparaissent en rouge (traction), les fibres opposées en bleu (compression) : c'est le diagramme des moments d'une poutre bi-encastree, dessiné par la simulation.

8 Étape 5 : endommagement

Le bloc `elif use_damage == 1` est la loi du §3.5, mot pour mot :

- `eps` : allongement principal maximal, lu sur la SVD du *nouveau* $\mathbf{F}$;
- `D_new` : interpolation linéaire entre ε_0 et ε_f , bornée à $[0, 1]$;
- `ti.max(D[p], D_new)` : irréversibilité.

Et D_p est consommé dans `P2G_solid` via `k = 1 - D[p]`. C'est tout : l'endommagement tient en quatre lignes.

Ce que l'on doit voir (touche 2). À charge 1, rien ne change ($\varepsilon < \varepsilon_0$, $D_{\max} = 0$ dans le panneau). À charge 2, les particules jaunissent aux encastrement (dessus) et à mi-portée (dessous), $D_{\max}$ monte vers 0.3 et la flèche augmente un peu : la poutre est fissurée mais tient. Ramenez la charge à 1 : la flèche diminue mais la couleur reste, D ne redescend jamais. À charge 3, $D_{\max}$ atteint 1 par endroits et la poutre s'affaisse sans se séparer : les particules de raideur nulle restent « accrochées » par la grille. C'est le problème annoncé au §3.6, et la raison de l'étape suivante.

9 Étape 6 : rupture

Deux morceaux de code. D'abord la *transition*, dans le bloc d'endommagement :

```

if use_rupture == 1 and D[p] >= 1.0:
    broken[p] = 1
    F_new = I

```

Ensuite le comportement *permanent* d'une particule rompue, en tête du `if` :

```

if broken[p] == 1:
    U, sig, V = ti.svd(F_new)
    for d in ti.static(range(2)):
        sig[d, d] = ti.math.clamp(sig[d, d], 0.1, 1.0)
    F_new = U @ sig @ V.transpose()

```

La boucle `for d in ti.static(range(2))` est déroulée à la compilation et écrête les deux allongements principaux; `U @ sig @ V.transpose()` recompose $\mathbf{F}$ avec la même rotation. Une particule rompue ne repasse jamais par le bloc d'endommagement (`elif`).

Ce que l'on doit voir (touche 3). Chargez progressivement. Vers 2.5 à 3, les premières particules rouges apparaissent *aux encastrement*, sur la fibre supérieure : c'est là que le moment fléchissant d'une poutre bi-encastree est maximal ($qL^2/12$ contre $qL^2/24$ à mi-portée). La poutre se détache

des piliers, devient une poutre sur appuis, puis se fissure à *mi-portée* par le dessous : trois rotules, mécanisme de ruine classique. Les deux moitiés tombent, tournent, heurtent le sol et s’y brisent en éclats (l’impact à plusieurs mètres par seconde sur une paroi rigide impose des déformations bien supérieures à ε_f).

Mesures faites sur ce code, avec les paramètres de la table du §3.7, sur 1 s :

Charge	Modèle	$D_{\max}$	Comportement
1	3	0	flèche ≈ 0.014 , aucune particule endommagée
2	3	0.28	fissuration visible, la poutre tient
3	3	1	rupture aux encastrement à $t \approx 0.1$ s, à mi-portée vers $t \approx 0.2$ s, chute
3	1	–	(élastique) flèche ≈ 0.04 , oscille, ne casse pas

10 Étape 7 : le programme de démonstration

MpM/demo_solid.py assemble le tout. Les particules sont placées sur un réseau régulier de 2×2 par cellule (plus régulier qu’un tirage aléatoire, ce qui compte pour un solide dont on regarde la forme) :

```
@ti.kernel
def init_beam(x, v, C, F, D, broken, x0: float, y0: float, n_px: int, spacing: float):
    for p in x:
        i = p % n_px
        j = p // n_px
        x[p] = [x0 + (i + 0.5) * spacing, y0 + (j + 0.5) * spacing]
        v[p] = [0.0, 0.0]
        C[p] = ti.Matrix.zero(ti.f32, 2, 2)
        F[p] = ti.Matrix.identity(ti.f32, 2)
        D[p] = 0.0
        broken[p] = 0
```

(les arguments `x`, `v`, ... sont des `ti.template()` dans le fichier ; on abrège ici.)

Un pas de temps, avec la charge `load` comme multiplicateur de gravité et les deux drapeaux :

```
def substep(load, use_damage, use_rupture):
    clear_grid()
    P2G_solid(grid_m, grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, mu_s, la_s, p_mass, p_vol)
    grid_update(grid_m, grid_v, mask, dt, load * g, bound, n_grid)
    G2P_solid(grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, bound, eps0, epsf, use_damage, use_rupture)
```

Affichage. On ne peut plus se contenter d’une image de la grille : il faut voir les particules solides et leur couleur. GGUI dessine des disques depuis un champ de positions avec `canvas.circles(x_s, radius=..., per_vertex_color=col_s)`, par-dessus l’image de fond (piliers en gris). Le kernel `solid_colors` remplit `col_s` : gris $\rightarrow$ jaune selon D , rouge si rompue ; ou bleu $\rightarrow$ rouge selon l’allongement principal (mode déformation). Le curseur de charge est un `gui.slider_float`. Le code complet est en annexe B.

Expériences.

- Matériau fragile : $\varepsilon_0 = 0.02$, $\varepsilon_f = 0.03$. La rupture devient brutale, sans phase jaune visible.
- Matériau ductile : $\varepsilon_f = 0.5$. La poutre pend comme du caoutchouc avant de céder.
- Appui simple : piliers arrêtés sous la poutre (`pos.y <= beam_y0`). La rupture se produit alors d’abord à mi-portée.
- Console : supprimez le pilier droit. Une poutre encastree-libre casse à l’encastrement.

- Critère de Von Mises : calculez $\sigma = \tau/J$ dans `G2P_solid`, puis σ_{eq} , et remplacez `eps` par σ_{eq}/E . Observez que les zones comprimées s'endommagent aussi.

Deuxième partie

Couplage avec le fluide APIC

11 Le principe : une grille, deux phases

11.1 Pourquoi c'est presque gratuit

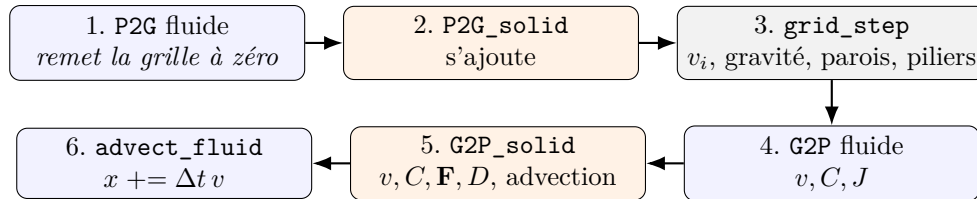
Le fluide dépose m_i , $(mv)_i$ et sa force de pression ; le solide dépose m_i , $(mv)_i$ et sa force élastique. Sur un nœud partagé,

$$m_i = m_i^f + m_i^s, \quad (mv)_i = (mv)_i^f + (mv)_i^s + \Delta t (\mathbf{f}_i^f + \mathbf{f}_i^s), \quad v_i = (mv)_i / m_i,$$

et chaque phase relit la *même* vitesse v_i . C'est un **couplage monolithique** : une seule équation de quantité de mouvement pour le mélange, résolue une fois par pas de temps. Il n'y a ni force de contact à calculer, ni itération entre les deux solveurs. Les conséquences physiques :

- **Pas d'interpénétration** : là où l'eau pousse le tablier, les nœuds portent les deux masses et une seule vitesse ; la pression du fluide accélère le solide et la raideur du solide freine le fluide. C'est exactement la réaction hydrodynamique voulue.
- **Adhérence à l'interface** : une vitesse commune signifie condition de *no-slip* entre eau et solide. Pour un tablier de pont c'est parfaitement acceptable.
- **Mélange sur une cellule** : l'interface n'est pas nette, elle a l'épaisseur du stencil. Un fluide et un solide qui se croisent à grande vitesse échangent un peu trop de quantité de mouvement. C'est la limite connue de cette approche ; le remède (deux champs de vitesse nodaux et une condition de contact entre eux) fait l'objet du dernier paragraphe.

11.2 L'ordre des kernels



- Le P2G du fluide remet la grille à zéro avant de déposer : il doit donc passer *en premier*, et P2G_solid (qui ne remet rien à zéro) s'ajoute ensuite. Aucune modification de `APIC/APIC.py` n'est nécessaire.
- `grid_step` du fluide sert tel quel : son masque `solid` (obstacles) devient le masque des piliers.
- Le G2P du fluide n'avance pas les particules (c'était le rôle de `time_integration`) ; un kernel `advect_fluid` de trois lignes le fait avec le *même* Δt .

11.3 Un seul pas de temps

Le fluide impose $\Delta t < \Delta x / c_f$ avec $c_f = \sqrt{E_f / \rho_f} = 20$, le solide $\Delta t < \Delta x / c_p$ avec $c_p \approx 45$. On prend

$$\Delta t = 0.4 \frac{\Delta x}{\max(c_f, c_p)} \approx 7 \cdot 10^{-5} \quad (n = 128),$$

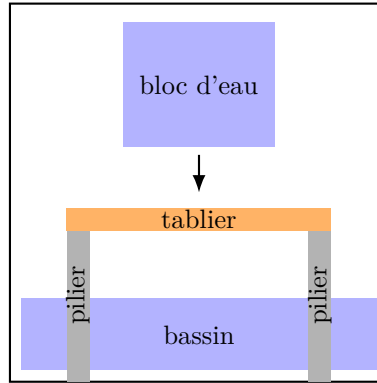
et on l'utilise *partout* : dans les deux P2G (facteur Δt devant les forces), dans les deux G2P (mise à jour de J et de $\mathbf{F}$) et dans les deux advections.

Remarque sur main.py. Dans le programme fluide actuel, le `dt` fixe passé à P2G et G2P n'est pas celui, adaptatif, que `time_integration` utilise pour avancer les particules. Les forces et l'advection ne sont alors pas intégrées avec le même pas, ce qui fausse J_p et la pression. Le solveur couplé corrige ce point en calculant un Δt unique à partir de la condition CFL *avant* la simulation. Si l'on tient à un pas adaptatif, il faut le calculer en début de sous-pas et le passer à tous les kernels.

12 Le programme couplé : main_fsi.py

12.1 Scène

Deux piliers (masque), un tablier encastré entre eux à mi-hauteur, un bassin au fond, et un bloc d'eau lâché au-dessus du tablier :



```
import taichi as ti
from APIC.APIC import P2G, G2P, grid_step
from Mpm.mpm_solid import (P2G_solid, G2P_solid, init_beam, solid_colors, solid_stats)

ti.init(arch=ti.gpu)

n_grid = 128
dx = 1.0 / n_grid
inv_dx = float(n_grid)
bound = 3
render_substep = 20

# fluide (comme main.py)
rho_f, E_f = 1.0, 400.0
p_vol_f = (0.5 * dx) ** 2
p_mass_f = p_vol_f * rho_f
water_x0, water_x1, water_y0, water_y1 = 0.30, 0.70, 0.62, 0.95 # bloc qui tombe
pool_x0, pool_x1, pool_y0, pool_y1 = 0.03, 0.97, 0.03, 0.22 # bassin
n_water = int(4 * (water_x1 - water_x0) * (water_y1 - water_y0) * n_grid * n_grid)
n_pool = int(4 * (pool_x1 - pool_x0) * (pool_y1 - pool_y0) * n_grid * n_grid)
n_fluid = n_water + n_pool

# solide (comme demo_solid.py)
rho_s, E_s, nu_s = 2.0, 3000.0, 0.3
mu_s = E_s / (2 * (1 + nu_s))
la_s = E_s * nu_s / ((1 + nu_s) * (1 - 2 * nu_s))
eps0, epsf = 0.05, 0.20
use_damage, use_rupture = 1, 1
p_spacing = 0.5 * dx
p_vol_s = p_spacing ** 2
p_mass_s = p_vol_s * rho_s
beam_x0, beam_x1, beam_y0, beam_y1 = 0.15, 0.85, 0.40, 0.46
```

```

pillar_w = 0.06
n_px = int(round((beam_x1 - beam_x0) / p_spacing))
n_py = int(round((beam_y1 - beam_y0) / p_spacing))
n_solid = n_px * n_py

# pas de temps commun
g = 9.81
c_f = (E_f / rho_f) ** 0.5
c_p = ((la_s + 2 * mu_s) / rho_s) ** 0.5
dt = 0.4 * dx / max(c_f, c_p)

```

12.2 Champs

Les champs du fluide portent le suffixe `_f`, ceux du solide `_s`; la grille est unique. Le masque s'appelle `solid` parce que c'est le nom attendu par `grid_step` (et il est en `ti.f32` comme dans `main.py`; la comparaison `== 1` fonctionne).

```

x_f = ti.Vector.field(2, ti.f32, n_fluid)
v_f = ti.Vector.field(2, ti.f32, n_fluid)
C_f = ti.Matrix.field(2, 2, ti.f32, n_fluid)
J_f = ti.field(ti.f32, n_fluid)

x_s = ti.Vector.field(2, ti.f32, n_solid)
v_s = ti.Vector.field(2, ti.f32, n_solid)
C_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
F_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
D_s = ti.field(ti.f32, n_solid)
broken_s = ti.field(ti.i32, n_solid)
col_s = ti.Vector.field(3, ti.f32, n_solid)

grid_v = ti.Vector.field(2, ti.f32, (n_grid, n_grid))
grid_m = ti.field(ti.f32, (n_grid, n_grid))
solid = ti.field(ti.f32, (n_grid, n_grid)) # 1 = pilier
img = ti.Vector.field(3, ti.f32, (n_grid, n_grid))

```

12.3 Initialisation et advection du fluide

```

@ti.kernel
def init_fluid():
    for p in x_f:
        if p < n_water:
            x_f[p] = [water_x0 + ti.random() * (water_x1 - water_x0),
                      water_y0 + ti.random() * (water_y1 - water_y0)]
        else:
            x_f[p] = [pool_x0 + ti.random() * (pool_x1 - pool_x0),
                      pool_y0 + ti.random() * (pool_y1 - pool_y0)]
        v_f[p] = [0.0, 0.0]
        C_f[p] = ti.Matrix.zero(ti.f32, 2, 2)
        J_f[p] = 1.0
    for i, j in solid:
        pos = ti.Vector([i, j]) * dx
        left = beam_x0 <= pos.x <= beam_x0 + pillar_w
        right = beam_x1 - pillar_w <= pos.x <= beam_x1
        solid[i, j] = 1.0 if ((left or right) and pos.y <= beam_y1) else 0.0

@ti.kernel
def advect_fluid():
    for p in x_f:

```

```

        x_f[p] += dt * v_f[p]
        x_f[p] = ti.math.clamp(x_f[p], bound * dx, 1.0 - bound * dx)

def init_scene():
    init_fluid()
    init_beam(x_s, v_s, C_s, F_s, D_s, broken_s, beam_x0, beam_y0, n_px, p_spacing)

```

12.4 Le pas de temps couplé

C'est le cœur de la partie II, et il tient en six appels dans l'ordre du schéma :

```

def substep():
    P2G(grid_m, grid_v, x_f, v_f, C_f, J_f, inv_dx, dt, dx, E_f, p_mass_f, p_vol_f)
    # 1
    P2G_solid(grid_m, grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, mu_s, la_s, p_mass_s, p_vol_s)
    # 2
    grid_step(grid_m, grid_v, solid, dt, g, bound, n_grid)
    # 3
    G2P(grid_m, grid_v, x_f, v_f, C_f, J_f, inv_dx, dt, dx, E_f, p_mass_f, p_vol_f)
    # 4
    G2P_solid(grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, bound, eps0, epsf, use_damage, use_rupture)
    # 5
    advect_fluid()
    # 6

```

Ce qui se passe sur un nœud sous le tablier quand l'eau arrive. Après l'appel 1, le nœud porte la masse et la quantité de mouvement de l'eau qui descend, plus la force de pression (l'eau se comprime à l'impact, $J_f < 1$, $\text{stress} > 0$). Après l'appel 2, il porte en plus la masse du tablier (au repos) et sa force élastique. L'appel 3 fait la moyenne pondérée par les masses : le nœud descend, moins vite que l'eau. À l'appel 4 l'eau relit cette vitesse : elle est freinée (et J_f diminue, la pression monte). À l'appel 5 le tablier relit la même vitesse : il est poussé vers le bas, son $\mathbf{F}$ se met à fléchir, et au pas suivant sa force élastique repoussera le nœud vers le haut. L'équilibre dynamique s'établit en quelques pas.

12.5 Affichage et boucle

Fond gris/bleu nuit depuis le masque, puis deux appels à `canvas.circles` : l'eau en bleu uni, le solide coloré par D ou par la déformation :

```

stats = solid_stats(D_s, broken_s)
solid_colors(F_s, D_s, broken_s, col_s, color_mode, epsf)
canvas.set_image(img)
canvas.circles(x_f, radius=0.5 * p_spacing, color=(0.35, 0.65, 1.0))
canvas.circles(x_s, radius=0.6 * p_spacing, per_vertex_color=col_s)

```

Le reste de la boucle (événements, panneau de texte) est celui de `main.py` ; voir l'annexe C.

12.6 Ce que l'on doit voir

Le bloc d'eau tombe pendant 0.2 s et frappe le tablier à environ 2 m/s. Le tablier fléchit (en mode déformation : rouge dessous à mi-portée, rouge dessus aux piliers), jaunit, puis vers $t \approx 0.3$ s les premières particules rouges apparaissent aux encastrement. Le tablier se décroche des deux piliers, tombe d'un bloc dans le bassin en entraînant l'eau qui le recouvrait, et coule (il est deux fois plus dense que l'eau). L'eau du bloc déferle sur les piliers et rejoint le bassin. Mesures sur ce code

($n = 128$, environ 20 000 particules de fluide et 2 700 de solide, 15 s de calcul par seconde simulée sur GPU) :

t (s)	$D_{\max}$	rompues	y mi-portée	commentaire
0.10	0	0	0.418	chute libre de l'eau, tablier au repos sous son poids
0.20	0.20	0	0.398	impact, flexion, début de l'endommagement
0.30	1	65	0.256	rupture aux deux encastresments, le tablier tombe
0.50	1	272	0.299	tablier dans le bassin, éclats aux extrémités
1.00	1	290	0.254	tablier au fond, l'eau se calme

La masse totale sur la grille (fluide + solide) est conservée à la précision machine, J_f reste dans $[0.65, 1.25]$ (les valeurs extrêmes sont des particules d'eau coincées entre le tablier qui tombe et un pilier) et la simulation reste stable indéfiniment. À $n = 256$ le programme tourne tel quel : Δt est divisé par deux automatiquement.

Expériences.

- Rendez le tablier plus résistant ($E_s = 10^4$ ou $\varepsilon_f = 0.4$) : il encaisse la vague et vibre. Plus fragile ($\varepsilon_f = 0.08$) : il se brise en plusieurs morceaux.
- Remplacez le bloc qui tombe par une rupture de barrage (colonne d'eau à gauche, `water_x0`, `water_x1` = 0.03, 0.30, du sol au sommet) : la vague frappe le pilier et passe sous le tablier ; observez la poussée vers le haut.
- Un bâtiment : plusieurs poutres et poteaux (appelez `init_beam` plusieurs fois sur des plages d'indices, ou généralisez-le à une liste de rectangles).

13 Limites et pour aller plus loin

1. **Deux champs de vitesse.** Pour une interface nette et un glissement eau/solide, on garde deux grilles, (m^f, v^f) et (m^s, v^s) , et l'on n'impose l'égalité des vitesses que dans la direction normale à l'interface, seulement si les deux phases se rapprochent (condition de contact). La normale se calcule comme le gradient de la masse solide sur la grille. C'est le schéma « multi-vitesses » de Bardenhagen et al. et le premier pas vers un vrai frottement.
2. **CPIC / fissures nettes.** Pour que deux fragments à moins d'une cellule ne se « voient » plus, chaque nœud porte deux vitesses (une par côté de la fissure) et les particules choisissent la leur selon une fonction de couleur.
3. **Plasticité.** Entre l'élastique et la rupture, un matériau réel plastifie. Le même écrêtage des s_k (borné à $[1 - \theta_c, 1 + \theta_s]$ au lieu de $[0.1, 1]$) donne la plasticité de la neige ; Drucker-Prager donne le sable.
4. **Réseau de liaisons.** Remplacer D_p par des liaisons B_{pq} entre particules voisines, cassées quand $\epsilon_{pq} > \epsilon_{\text{crit}}$, rend la fissure discrète et orientée. Il faut une liste de voisins (fixée à l'initialisation) et une force de rappel $\mathbf{f}_{pq}$ le long de chaque liaison, en plus ou à la place de la contrainte MPM.
5. **3D.** Comme pour le fluide : vecteurs à trois composantes, matrices 3×3 , stencil $3 \times 3 \times 3$; `ti.svd` fonctionne en 3D et les formules ne changent pas.

A Code complet : MpM/mpm_solid.py

```
# MpM/mpm_solid.py -- Solide MLS-MPM 2D : élasticité corotationnelle + endommagement +
# rupture
#
# Même grille et mêmes poids B-spline quadratiques que le fluide APIC (APIC/APIC.py).
# Chaque particule solide porte :
```

```

# x, v, C : position, vitesse, matrice affine APIC (comme le fluide)
# F       : gradient de déformation (2x2), F = I au repos
# D       : variable d'endommagement dans [0, 1] (0 = intact, 1 = rompu)
# broken  : 1 si la particule a perdu toute cohésion (rupture)

import taichi as ti

@ti.func
def kirchhoff_stress(F, mu: float, la: float):
    """tau = P F^T du modèle corotationnel : 2 mu (F - R) F^T + la J (J - 1) I."""
    U, sig, V = ti.svd(F)
    R = U @ V.transpose()
    J = F.determinant()
    I = ti.Matrix.identity(ti.f32, 2)
    return 2.0 * mu * (F - R) @ F.transpose() + la * J * (J - 1.0) * I

@ti.kernel
def P2G_solid(grid_m: ti.template(), grid_v: ti.template(),
              x: ti.template(), v: ti.template(), C: ti.template(), F: ti.template(),
              D: ti.template(), broken: ti.template(),
              inv_dx: float, dt: float, dx: float,
              mu: float, la: float, p_mass: float, p_vol: float):
    """Ne remet PAS la grille à zéro : on peut l'appeler après le P2G du fluide
    (grille partagée)."""
    for p in x:
        base = (x[p] * inv_dx - 0.5).cast(int)
        fx = x[p] * inv_dx - base
        w = [0.5 * (1.5 - fx)**2,
              0.75 - (fx - 1.0)**2,
              0.5 * (fx - 0.5)**2]

        k = 1.0 - D[p]
        if broken[p] == 1:
            k = 1.0
        tau = kirchhoff_stress(F[p], k * mu, k * la)

        affine = (-dt * p_vol * 4.0 * inv_dx * inv_dx) * tau + p_mass * C[p]

        for i, j in ti.static(ti.ndrange(3, 3)):
            offset = ti.Vector([i, j])
            d_pos = (offset - fx) * dx
            weight = w[i].x * w[j].y
            grid_v[base + offset] += weight * (p_mass * v[p] + affine @ d_pos)
            grid_m[base + offset] += weight * p_mass

@ti.kernel
def grid_update(grid_m: ti.template(), grid_v: ti.template(), mask: ti.template(),
               dt: float, g: float, bound: int, n_grid: int):
    for i, j in grid_m:
        if grid_m[i, j] > 0:
            grid_v[i, j] /= grid_m[i, j]
            grid_v[i, j].y -= dt * g
            if i < bound and grid_v[i, j].x < 0:
                grid_v[i, j].x = 0.0
            if i > n_grid - bound and grid_v[i, j].x > 0:
                grid_v[i, j].x = 0.0
            if j < bound and grid_v[i, j].y < 0:

```

```

        grid_v[i, j].y = 0.0
    if j > n_grid - bound and grid_v[i, j].y > 0:
        grid_v[i, j].y = 0.0
    if mask[i, j] == 1:
        grid_v[i, j] = [0.0, 0.0]

@ti.kernel
def G2P_solid(grid_v: ti.template(),
              x: ti.template(), v: ti.template(), C: ti.template(), F: ti.template(),
              D: ti.template(), broken: ti.template(),
              inv_dx: float, dt: float, dx: float, bound: int,
              eps0: float, epsf: float, use_damage: int, use_rupture: int):
    I = ti.Matrix.identity(ti.f32, 2)
    for p in x:
        base = (x[p] * inv_dx - 0.5).cast(int)
        fx = x[p] * inv_dx - base
        w = [0.5 * (1.5 - fx)**2,
              0.75 - (fx - 1.0)**2,
              0.5 * (fx - 0.5)**2]

        v_new = ti.Vector.zero(ti.f32, 2)
        C_new = ti.Matrix.zero(ti.f32, 2, 2)
        for i, j in ti.static(ti.ndrange(3, 3)):
            offset = ti.Vector([i, j])
            d_pos = (offset - fx) * dx
            weight = w[i].x * w[j].y
            g_v = grid_v[base + offset]
            v_new += weight * g_v
            C_new += weight * g_v.outer_product(d_pos) * (4.0 * inv_dx * inv_dx)
        v[p] = v_new
        C[p] = C_new

        F_new = (I + dt * C_new) @ F[p]

        if broken[p] == 1:
            U, sig, V = ti.svd(F_new)
            for d in ti.static(range(2)):
                sig[d, d] = ti.math.clamp(sig[d, d], 0.1, 1.0)
            F_new = U @ sig @ V.transpose()

        elif use_damage == 1:
            U, sig, V = ti.svd(F_new)
            eps = ti.max(sig[0, 0], sig[1, 1]) - 1.0
            D_new = ti.math.clamp((eps - eps0) / (epsf - eps0), 0.0, 1.0)
            D[p] = ti.max(D[p], D_new)
            if use_rupture == 1 and D[p] >= 1.0:
                broken[p] = 1
                F_new = I

        F[p] = F_new
        x[p] += dt * v[p]
        x[p] = ti.math.clamp(x[p], bound * dx, 1.0 - bound * dx)

@ti.kernel
def init_beam(x: ti.template(), v: ti.template(), C: ti.template(), F: ti.template(),
              D: ti.template(), broken: ti.template(),
              x0: float, y0: float, n_px: int, spacing: float):
    for p in x:

```

```

        i = p % n_px
        j = p // n_px
        x[p] = [x0 + (i + 0.5) * spacing, y0 + (j + 0.5) * spacing]
        v[p] = [0.0, 0.0]
        C[p] = ti.Matrix.zero(ti.f32, 2, 2)
        F[p] = ti.Matrix.identity(ti.f32, 2)
        D[p] = 0.0
        broken[p] = 0

@ti.kernel
def solid_colors(F: ti.template(), D: ti.template(), broken: ti.template(),
                col: ti.template(), mode: int, eps_scale: float):
    for p in D:
        if mode == 0:
            if broken[p] == 1:
                col[p] = ti.Vector([0.90, 0.15, 0.15])
            else:
                col[p] = ti.Vector([0.75, 0.75, 0.75]) * (1 - D[p]) + ti.Vector([1.0,
0.85, 0.1]) * D[p]
            else:
                U, sig, V = ti.svd(F[p])
                eps = ti.max(sig[0, 0], sig[1, 1]) - 1.0
                t = ti.math.clamp(0.5 + 0.5 * eps / eps_scale, 0.0, 1.0)
                col[p] = ti.Vector([0.2, 0.4, 1.0]) * (1 - t) + ti.Vector([1.0, 0.25,
0.1]) * t

@ti.kernel
def solid_stats(D: ti.template(), broken: ti.template()) -> ti.types.vector(2, ti.f32):
    n_broken = 0
    D_max = 0.0
    for p in D:
        n_broken += broken[p]
        ti.atomic_max(D_max, D[p])
    return ti.Vector([float(n_broken), D_max])

```

B Code complet : MpM/demo_solid.py

```

# MpM/demo_solid.py -- Partie I : une poutre MPM seule (sans fluide)
# Lancer depuis la racine du projet : python -m MpM.demo_solid
# Touches : 1/2/3 modèle, ESPACE couleur, R reset, ESC quitter. Curseur "charge" =
multiple de g.

import taichi as ti

try:
    from MpM.mpm_solid import *
except ImportError:
    # lancé directement depuis le dossier MpM/
    from mpm_solid import *

ti.init(arch=ti.gpu)

n_grid = 128
dx = 1.0 / n_grid
inv_dx = float(n_grid)
bound = 3
render_substep = 20

```

```

rho_s = 2.0
E_s = 3000.0
nu_s = 0.3
mu_s = E_s / (2 * (1 + nu_s))
la_s = E_s * nu_s / ((1 + nu_s) * (1 - 2 * nu_s))
g = 9.81

eps0 = 0.05                                # début de l'endommagement (allongement principal)
epsf = 0.20                                # rupture complète (petit = fragile, grand =
    ductile)

c_p = ((la_s + 2 * mu_s) / rho_s) ** 0.5
dt = 0.4 * dx / c_p

p_spacing = 0.5 * dx                        # 2 x 2 particules par cellule
p_vol = p_spacing ** 2
p_mass = p_vol * rho_s

beam_x0, beam_x1 = 0.15, 0.85
beam_y0, beam_y1 = 0.55, 0.61
pillar_w = 0.06

n_px = int(round((beam_x1 - beam_x0) / p_spacing))
n_py = int(round((beam_y1 - beam_y0) / p_spacing))
n_solid = n_px * n_py

x_s = ti.Vector.field(2, ti.f32, n_solid)
v_s = ti.Vector.field(2, ti.f32, n_solid)
C_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
F_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
D_s = ti.field(ti.f32, n_solid)
broken_s = ti.field(ti.i32, n_solid)
col_s = ti.Vector.field(3, ti.f32, n_solid)

grid_v = ti.Vector.field(2, ti.f32, (n_grid, n_grid))
grid_m = ti.field(ti.f32, (n_grid, n_grid))
mask = ti.field(ti.i32, (n_grid, n_grid))
img = ti.Vector.field(3, ti.f32, (n_grid, n_grid))

@ti.kernel
def init_mask():
    for i, j in mask:
        pos = ti.Vector([i, j]) * dx
        left = beam_x0 <= pos.x <= beam_x0 + pillar_w
        right = beam_x1 - pillar_w <= pos.x <= beam_x1
        mask[i, j] = 1 if ((left or right) and pos.y <= beam_y1) else 0

@ti.kernel
def clear_grid():
    for i, j in grid_m:
        grid_v[i, j] = [0.0, 0.0]
        grid_m[i, j] = 0.0

@ti.kernel
def render_background():
    for i, j in img:

```

```

        img[i, j] = ti.Vector([0.35, 0.35, 0.35]) if mask[i, j] == 1 else
ti.Vector([0.02, 0.02, 0.08])

def init_scene():
    init_beam(x_s, v_s, C_s, F_s, D_s, broken_s, beam_x0, beam_y0, n_px, p_spacing)
    init_mask()

def substep(load, use_damage, use_rupture):
    clear_grid()
    P2G_solid(grid_m, grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, mu_s, la_s, p_mass, p_vol)
    grid_update(grid_m, grid_v, mask, dt, load * g, bound, n_grid)
    G2P_solid(grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
              inv_dx, dt, dx, bound, eps0, epsf, use_damage, use_rupture)

def main():
    window = ti.ui.Window("MPM 2D - poutre : élasticité, endommagement, rupture",
res=(700, 700))
    canvas = window.get_canvas()
    gui = window.get_gui()
    model = 3
    color_mode = 0
    load = 1.0
    t_sim = 0.0
    init_scene()
    render_background()

    while window.running:
        while window.get_event(ti.ui.PRESS):
            k = window.event.key
            if k == ti.ui.ESCAPE:
                window.running = False
            elif k == ti.ui.SPACE:
                color_mode = 1 - color_mode
            elif k == 'r':
                init_scene()
                t_sim = 0.0
            elif k in ('1', '2', '3'):
                model = int(k)

        use_damage = 1 if model >= 2 else 0
        use_rupture = 1 if model == 3 else 0
        for _ in range(render_substep):
            substep(load, use_damage, use_rupture)
            t_sim += dt

        stats = solid_stats(D_s, broken_s)
        solid_colors(F_s, D_s, broken_s, col_s, color_mode, epsf)
        canvas.set_image(img)
        canvas.circles(x_s, radius=0.6 * p_spacing, per_vertex_color=col_s)

        gui.begin("MPM solide", 0.02, 0.02, 0.46, 0.30)
        load = gui.slider_float("charge (x g)", load, 0.0, 6.0)
        gui.text(f"modele : [{',', 'élastique', 'endommagement', 'endommagement +
rupture'}[model]] (touches 1/2/3)")
        gui.text(f"dt = {dt:.2e} t = {t_sim:.3f} s")
        gui.text(f"D max = {stats[1]:.2f} rompues = {int(stats[0])}/{n_solid}")

```

```

        gui.text("couleur : " + ("endommagement" if color_mode == 0 else "deformation
principale") + " (ESPACE)")
        gui.text("R : reset      ESC : quitter")
        gui.end()
        window.show()

if __name__ == "__main__":
    main()

```

C Code complet : main_fsi.py

```

# main_fsi.py -- Partie II : eau APIC + pont MPM sur une grille commune
# Touches : ESPACE couleur du solide, R reset, ESC quitter.

import taichi as ti
from APIC.APIC import P2G, G2P, grid_step
from MpM.mpm_solid import (P2G_solid, G2P_solid, init_beam, solid_colors, solid_stats)

ti.init(arch=ti.gpu)

n_grid = 128
dx = 1.0 / n_grid
inv_dx = float(n_grid)
bound = 3
render_substep = 20

# fluide (comme main.py)
rho_f = 1.0
E_f = 400.0
p_vol_f = (0.5 * dx) ** 2
p_mass_f = p_vol_f * rho_f

water_x0, water_x1, water_y0, water_y1 = 0.30, 0.70, 0.62, 0.95
pool_x0, pool_x1, pool_y0, pool_y1 = 0.03, 0.97, 0.03, 0.22
n_water = int(4 * (water_x1 - water_x0) * (water_y1 - water_y0) * n_grid * n_grid)
n_pool = int(4 * (pool_x1 - pool_x0) * (pool_y1 - pool_y0) * n_grid * n_grid)
n_fluid = n_water + n_pool

# solide (comme MpM/demo_solid.py)
rho_s = 2.0
E_s = 3000.0
nu_s = 0.3
mu_s = E_s / (2 * (1 + nu_s))
la_s = E_s * nu_s / ((1 + nu_s) * (1 - 2 * nu_s))
eps0, epsf = 0.05, 0.20
use_damage, use_rupture = 1, 1

p_spacing = 0.5 * dx
p_vol_s = p_spacing ** 2
p_mass_s = p_vol_s * rho_s

beam_x0, beam_x1 = 0.15, 0.85
beam_y0, beam_y1 = 0.40, 0.46
pillar_w = 0.06
n_px = int(round((beam_x1 - beam_x0) / p_spacing))
n_py = int(round((beam_y1 - beam_y0) / p_spacing))
n_solid = n_px * n_py

```

```

# pas de temps commun
g = 9.81
c_f = (E_f / rho_f) ** 0.5
c_p = ((la_s + 2 * mu_s) / rho_s) ** 0.5
dt = 0.4 * dx / max(c_f, c_p)

# champs fluide
x_f = ti.Vector.field(2, ti.f32, n_fluid)
v_f = ti.Vector.field(2, ti.f32, n_fluid)
C_f = ti.Matrix.field(2, 2, ti.f32, n_fluid)
J_f = ti.field(ti.f32, n_fluid)

# champs solide
x_s = ti.Vector.field(2, ti.f32, n_solid)
v_s = ti.Vector.field(2, ti.f32, n_solid)
C_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
F_s = ti.Matrix.field(2, 2, ti.f32, n_solid)
D_s = ti.field(ti.f32, n_solid)
broken_s = ti.field(ti.i32, n_solid)
col_s = ti.Vector.field(3, ti.f32, n_solid)

# grille commune
grid_v = ti.Vector.field(2, ti.f32, (n_grid, n_grid))
grid_m = ti.field(ti.f32, (n_grid, n_grid))
solid = ti.field(ti.f32, (n_grid, n_grid)) # 1 = pilier
img = ti.Vector.field(3, ti.f32, (n_grid, n_grid))

@ti.kernel
def init_fluid():
    for p in x_f:
        if p < n_water:
            x_f[p] = [water_x0 + ti.random() * (water_x1 - water_x0),
                      water_y0 + ti.random() * (water_y1 - water_y0)]
        else:
            x_f[p] = [pool_x0 + ti.random() * (pool_x1 - pool_x0),
                      pool_y0 + ti.random() * (pool_y1 - pool_y0)]
        v_f[p] = [0.0, 0.0]
        C_f[p] = ti.Matrix.zero(ti.f32, 2, 2)
        J_f[p] = 1.0
    for i, j in solid:
        pos = ti.Vector([i, j]) * dx
        left = beam_x0 <= pos.x <= beam_x0 + pillar_w
        right = beam_x1 - pillar_w <= pos.x <= beam_x1
        solid[i, j] = 1.0 if ((left or right) and pos.y <= beam_y1) else 0.0

@ti.kernel
def advect_fluid():
    for p in x_f:
        x_f[p] += dt * v_f[p]
        x_f[p] = ti.math.clamp(x_f[p], bound * dx, 1.0 - bound * dx)

@ti.kernel
def render_background():
    for i, j in img:
        img[i, j] = ti.Vector([0.35, 0.35, 0.35]) if solid[i, j] == 1 else
ti.Vector([0.02, 0.02, 0.08])

```

```

def init_scene():
    init_fluid()
    init_beam(x_s, v_s, C_s, F_s, D_s, broken_s, beam_x0, beam_y0, n_px, p_spacing)

def substep():
    P2G(grid_m, grid_v, x_f, v_f, C_f, J_f, inv_dx, dt, dx, E_f, p_mass_f, p_vol_f)
    # 1
    P2G_solid(grid_m, grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
               inv_dx, dt, dx, mu_s, la_s, p_mass_s, p_vol_s)
    # 2
    grid_step(grid_m, grid_v, solid, dt, g, bound, n_grid)
    # 3
    G2P(grid_m, grid_v, x_f, v_f, C_f, J_f, inv_dx, dt, dx, E_f, p_mass_f, p_vol_f)
    # 4
    G2P_solid(grid_v, x_s, v_s, C_s, F_s, D_s, broken_s,
               inv_dx, dt, dx, bound, eps0, epsf, use_damage, use_rupture)
    # 5
    advect_fluid()
    # 6

def main():
    window = ti.ui.Window("APIC + MPM : eau et pont", res=(700, 700))
    canvas = window.get_canvas()
    gui = window.get_gui()
    color_mode = 0
    t_sim = 0.0
    init_scene()
    render_background()

    while window.running:
        while window.get_event(ti.ui.PRESS):
            k = window.event.key
            if k == ti.ui.ESCAPE:
                window.running = False
            elif k == ti.ui.SPACE:
                color_mode = 1 - color_mode
            elif k == 'r':
                init_scene()
                t_sim = 0.0

        for _ in range(render_substep):
            substep()
            t_sim += dt

        stats = solid_stats(D_s, broken_s)
        solid_colors(F_s, D_s, broken_s, col_s, color_mode, epsf)
        canvas.set_image(img)
        canvas.circles(x_f, radius=0.5 * p_spacing, color=(0.35, 0.65, 1.0))
        canvas.circles(x_s, radius=0.6 * p_spacing, per_vertex_color=col_s)

        gui.begin("APIC + MPM", 0.02, 0.02, 0.46, 0.22)
        gui.text(f"dt = {dt:.2e}    t = {t_sim:.3f} s")
        gui.text(f"D max = {stats[1]:.2f}    rompues = {int(stats[0])}/{n_solid}")
        gui.text("couleur : " + ("endommagement" if color_mode == 0 else "deformation
principale") + "    (ESPACE)")
        gui.text("R : reset    ESC : quitter")

```

```
        gui.end()  
        window.show()  
  
if __name__ == "__main__":  
    main()
```